PRODUCT DESIGN & DELIVERY

School ERP UI Architecture

Dashboard concept and staged frontend implementation plan



CURRENT SCOPE	DESIGN DIRECTION
School and branches · Classes and sections · Student records · Paid/unpaid fee status	A calm, responsive administrator workspace with fast access to rosters and fee follow-up.

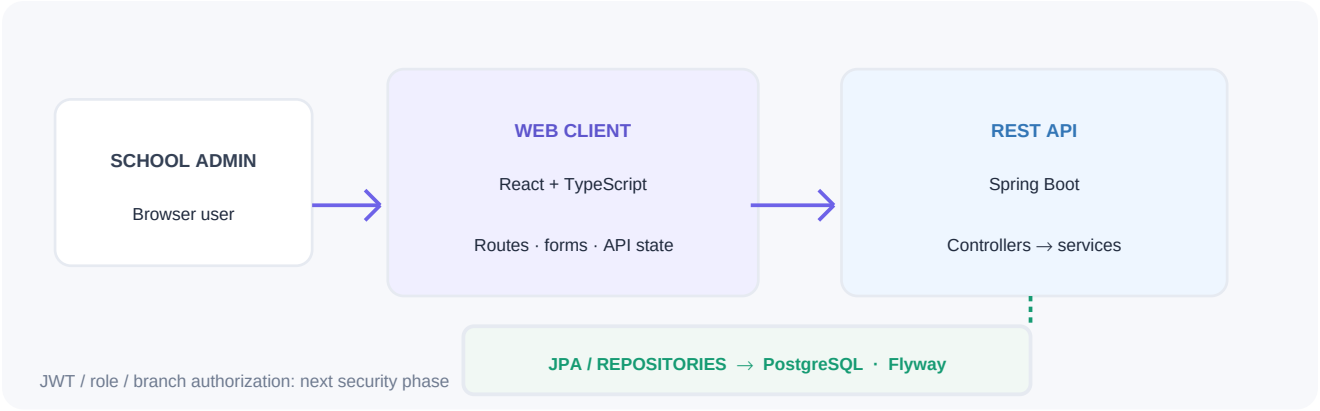
Prepared 3 November 2026 · Prototype uses synthetic sample records

1. Product direction

Build a responsive administrator workspace for a single school installation with one or more branches. The core workflow is to set up branches, create classes and sections, register students in the correct placement, then review and update fee status.

The first release focuses on administration. Attendance, staff, examinations, payment transactions, and parent portals remain future modules. The included browser prototype demonstrates Overview, Students, Classes & sections, and Fee status using synthetic in-browser data; it is not API-connected.

2. Proposed application architecture



Production direction

Use a separate React + TypeScript client with typed API calls to the versioned Spring REST API. Keep view rendering, data fetching, and business rules distinct. The Spring service layer remains authoritative for branch/class/section placement and fee status changes.

Frontend feature structure

Area	Responsibilities
app/	Routes, shared layout, navigation, session state
components/	Tables, forms, status pills, dialogs, loading and error states
features/dashboard/	Overview metrics, recent students, quick actions
features/students/	Roster, search/filters, create/edit, class and section roster
features/classes/	Branch class list and sections under each class
features/fees/	Paid/unpaid summary, unpaid list, status update
lib/ and types/	API client, query cache, formatters, request/response types

3. Pages and API mapping

Page	Primary content	API calls
Overview	Student, fee, class, and section counts; recent students; shortcuts	GET /students · /students/fees/summary · classes by branch
Students	Search, class/section filters, roster, create/edit	GET/POST /students · GET/PUT /students/{id} · status PATCH
Classes & sections	Class list by branch; sections within a class; create/update/deactivate	/classes · /sections?classId={id} · their update/status routes
Class roster	Students in a class or section	GET /classes/{classId}/students · /classes/{classId}/sections/{sectionId}/students
Fee status	Paid/unpaid totals, unpaid roster, status action	GET /students/fees/summary · /students/fees/unpaid · PATCH /students/{id}/fee-status

Key interaction rules

In student entry, load classes after branch selection and sections after class selection. Display class and section names in tables instead of raw IDs. Give field-level validation and actionable placement errors. On a fee status change, wait for the server response before updating the count and list; show a confirmation or undo affordance where appropriate.

Responsive design

Use a shared shell with left navigation, school/branch context, page title, action area, and content region. Collapse the navigation into a menu on small screens. Tables should scroll horizontally or switch to stacked cards on phones. Use visible labels, keyboard focus, readable contrast, and status text as well as color.

Current limits and release prerequisites

Fee status currently means a simple PAID or UNPAID field; there are no amounts, due dates, receipts, or payment transactions. The current security configuration permits all requests for development. Add login, JWT, role checks, and branch authorization before exposing real student data. The prototype uses synthetic data and local interactions only.

4. Phased implementation plan

0 · API and data contract

Confirm student, class, section, fee summary, and roster response shapes. Review OpenAPI. Add server pagination/filtering before production-scale lists; decide whether inactive students appear in metrics.

Exit: Done when UI page requirements map to stable, documented API contracts.

1 · Frontend shell

Create React + TypeScript app, design tokens, responsive navigation, typed API client, and loading/empty/error patterns.

Exit: Done when routes render consistently on phone, tablet, and desktop.

2 · Classes and sections

Build class and section lists and create/edit/deactivate flows. Surface duplicate and missing-parent errors.

Exit: Done when admins can build academic structure without raw IDs.

3 · Student administration

Build search, class/section filters, rosters, dependent selectors, create/edit forms, and active status control.

Exit: Done when student CRUD and class/section rosters work through the API.

4 · Fee status

Build paid/unpaid summary and unpaid list. Add confirmation and update UI only after API success.

Exit: Done when fee status changes and totals remain in sync with the server.

5 · Security

Replace allow-all security with login/token validation, role permissions, branch-scoped access, protected routes, and sign-out.

Exit: Done when anonymous and cross-branch access are rejected appropriately.

6 · Quality and release

Add component/API tests, accessibility and responsive checks, environment API configuration, build/deployment notes, and current docs.

Exit: Done when core flows have repeatable checks and production configuration is documented.

Suggested delivery order

Deliver one usable vertical slice at a time: (1) shell plus classes/sections, (2) student registration and rosters, (3) fee summary and unpaid follow-up, (4) authentication and branch authorization, then (5) quality hardening and deployment. Keep the static prototype as a visual reference until the production client replaces it.